Firestore Database-Access Reference

MedicRecall / Flashcard — Cloud Firestore usage guide

Test login

Use this login if you want to edit things on the MedicRecall app and test them. Try not to play with already existing databse functionality unless necessary. Or if you just want to login to the app and explore how it works at the moment.

`samchimaster@gmail.com`
`medicalReE345!`

Example: the `topics` collection

Here's a quick example of what a document looks like and how to read and edit it. The `topics` collection gets used for the flashcards and for building the study calendar. Each document is one topic with a random id (e.g. `86CmER3DedplPCH6tg7C`).

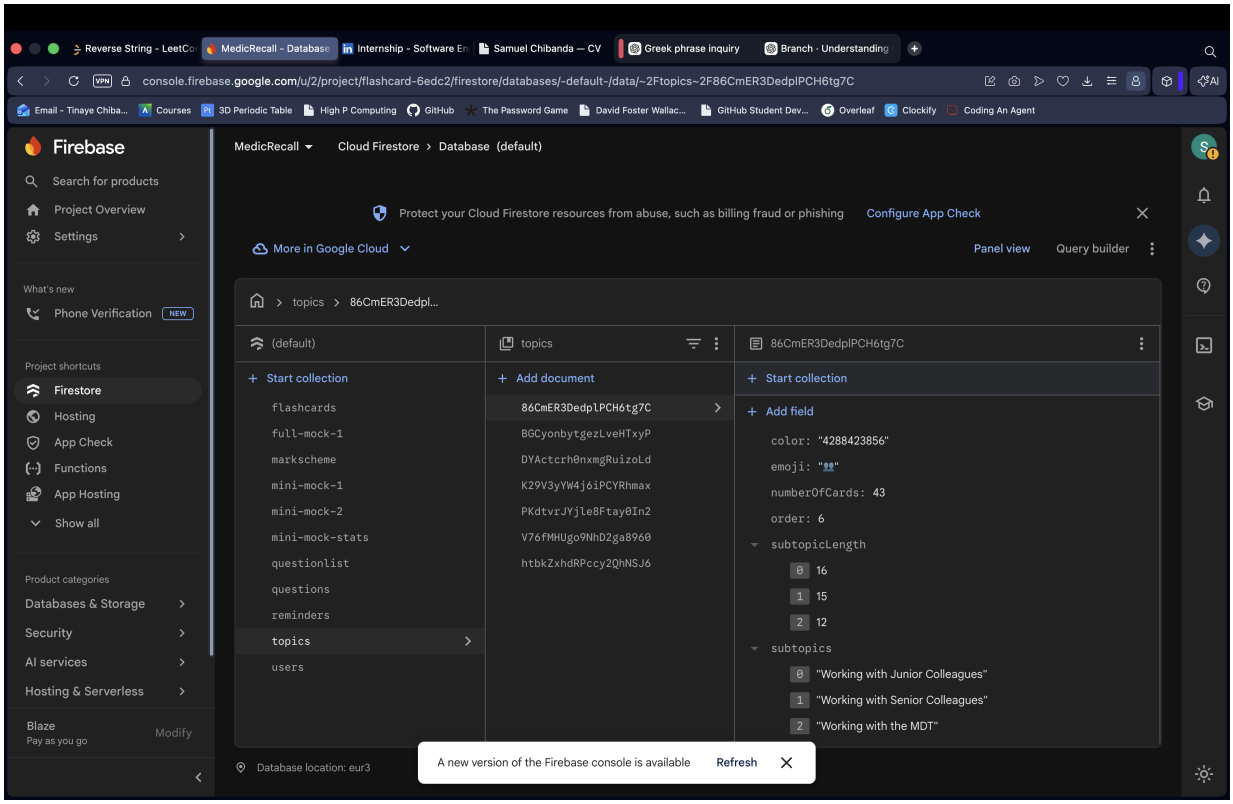


Figure 1: The `topics` collection in the Firebase console, with one topic open and its fields on the right.

A topic has these fields:

- `color`, `emoji` — just for display.

- `numberOfCards` — how many cards are in the topic.
- `order` — where it shows up in lists.
- `subtopics` — list of subtopic names.
- `subtopicLength` — how many cards each subtopic has. Same order as `subtopics`, so `subtopics[0]` has `subtopicLength[0]` cards.

Read all the topics. It's a small collection so we just grab everything:

```
QuerySnapshot snapshot =
    await FirebaseFirestore.instance.collection('topics').get();
for (var doc in snapshot.docs) {
  Map<String, dynamic> data = doc.data() as Map<String, dynamic>;
  String emoji = data['emoji'];
  int cards = data['numberOfCards'];
  List<dynamic> subtopics = data['subtopics'];      // ["Working with...", ...]
  List<dynamic> lengths   = data['subtopicLength']; // [16, 15, 12]
}
```

Read them in order. Add `orderBy` on the `order` field:

```
QuerySnapshot snapshot = await FirebaseFirestore.instance
    .collection('topics')
    .orderBy('order')
    .get();
```

Change one field. Grab the document by its id and `update` the field. Everything else stays as it is:

```
await FirebaseFirestore.instance
    .collection('topics')
    .doc('86CmER3DedplPCH6tg7C')
    .update({'numberOfCards': 44});
```

Change a list (e.g. add a subtopic). You can't edit one item in a list, so read it, change it, and write the whole thing back. Remember to keep `subtopics` and `subtopicLength` matching:

```
final ref = FirebaseFirestore.instance
    .collection('topics').doc('86CmER3DedplPCH6tg7C');

DocumentSnapshot snap = await ref.get();
List<dynamic> subtopics      = snap['subtopics'];
List<dynamic> subtopicLength = snap['subtopicLength'];

subtopics.add('Working with Patients'); // new subtopic
subtopicLength.add(10);                 // its card count, same spot

await ref.update({
  'subtopics': subtopics,
  'subtopicLength': subtopicLength,
});
```

Add a new topic. Use `add` and Firebase makes the id for you:

```
await FirebaseFirestore.instance.collection('topics').add({
  'color': '4288423856',
  'emoji': '\u{1F465}',
  'numberOfCards': 0,
  'order': 7,
  'subtopics': <String>[],
  'subtopicLength': <int>[],
});
```

That's basically it — `get`, `update`, `add`. The mock pages below do the same things, just with bigger documents.

1 The Mock pages

1.1 How it fits together

The mocks are spread over a few screens that all share one widget:

- `lib/widgets/minimock_exam.dart` — the actual exam page. It's an abstract widget (`MiniMockExam`) used by both the mini-mocks and the full mock. You pass it a `questionDirectory` (the collection the questions live in) and an `answerDirectory` (the document name to save the answers under). This is where questions are read and answers are saved.
- `lib/screens/minimock_landing.dart` — the start page; reads the exam date to see which mocks are unlocked.
- `lib/screens/minimock_review.dart` — the review/flag page; when you finish it writes a "Completed" marker onto your calendar.
- `lib/screens/minimock_scoring.dart` — reads the questions, your saved answers and the average score, works out your score and saves it.
- `lib/screens/minimock_feedback.dart` — reads the questions and your answers again to show per-question feedback.

The full mock screens (`fullmock_landing.dart`, `fullmock_review.dart`) use the same widget with different `questionDirectory` / `answerDirectory` values, so they're not shown separately.

One thing you'll see everywhere: to get a user's document we don't look it up by id, we search the `users` collection for the one whose `userID` field matches the logged-in user. Then we work with documents under that.

1.2 The exam page: read questions, save answers

`lib/widgets/minimock_exam.dart`

Read the questions. They aren't stored in order in Firebase, so we sort by a `questionNumber` field. The collection name is passed in, which is what lets one widget run every mock.

```
// Fetch mini-mock questions from firestore and store as a list of maps
Future<List<Map<String, dynamic>>> queryQuestionData() async {
  // questions are not properly stored in order on firestore so order via a field instead
  QuerySnapshot snapshot = await FirebaseFirestore.instance
      .collection(widget.questionDirectory)
      .orderBy('questionNumber')
      .get();
  return snapshot.docs.map((doc) => doc.data() as Map<String, dynamic>).toList();
}
```

Save the answers. When time runs out or the user submits, `updateExamResults()` runs. It finds the user's document (the `userID` search), then `sets` a document in their `examAnswers` with the answers and a timestamp.

```
Future<void> updateExamResults(List<List<String>> answers) async {
  Map<String, List<String>> answerMap = {
    for (int i = 0; i < questions.length; i++) '${i+1}' : answers[i]
  };
  final CollectionReference users =
      FirebaseFirestore.instance.collection('users');

  // find this user's document
  QuerySnapshot u = await users.where("userID", isEqualTo: user.uid).get();
  var id = u.docs.last.id;

  DocumentReference userDocRef =
      FirebaseFirestore.instance.collection('users').doc(id);
  DocumentReference examAnswersDocRef =
      userDocRef.collection('examAnswers').doc(widget.answerDirectory);

  // save the answers
  await examAnswersDocRef.set({
    'answers': answerMap,
    'timestamp': FieldValue.serverTimestamp(),
  });
}
```

A couple of things worth knowing: `set()` overwrites the whole document (use `SetOptions(merge: true)` if you only want to change some fields), and `FieldValue.serverTimestamp()` uses Firebase's clock instead of the phone's.

1.3 Landing page: read the exam date

`lib/screens/minimock_landing.dart`

This reads the user's `TestDay` (and `firstStudyDay`) to decide which mocks are unlocked. Same `userID` search as above. Watch out for the `Timestamp` to `DateTime` conversion — it catches people out.

```
final CollectionReference _userCollectionRef =
    FirebaseFirestore.instance.collection('users');

// Function from calendar_screen
Future<void> getTestDay() async {
  QuerySnapshot querySnapshot =
      await _userCollectionRef.where("userID", isEqualTo: user.uid).get();

  var userFromDb = querySnapshot.docs.first;
  Map<String, dynamic> data = userFromDb.data() as Map<String, dynamic>;
  if (data.containsKey("TestDay") || data["TestDay"] != null) {
    // firebase is weird. conversion from TimeStamp to DateTime object
    _examDate = DateTime.fromMicrosecondsSinceEpoch(
        userFromDb["TestDay"].microsecondsSinceEpoch);
    setState(() { examSet = true; });
  } else {
    setState(() { examSet = false; });
  }
}
```

`getFirstStudyDay()` does the same thing for `firstStudyDay`, but converts with the cleaner `(data["firstStudyDay"] as Timestamp).toDate()` — use that style.

1.4 Review page: updating calendar events

`lib/screens/minimock_review.dart`

When a mock is done, this marks the calendar entry as "Completed". First it finds the user and opens their `events`:

```
Future<void> userData() async {
  QuerySnapshot querySnapshot =
      await users.where("userID", isEqualTo: user.uid).get();
  setState(() { id = querySnapshot.docs.last.id; });

  _eventsCollectionRef =
      FirebaseFirestore.instance.collection('users/$id/events');
  var snapshots = await _eventsCollectionRef.get();
  // ... go through the docs, reading each one's 'date' and 'events'
}
```

To save, it looks for an event on that date: if one exists it `updates` it, otherwise it `adds` a new one. The same code shows up in `calendar_screen.dart`, `dashboard_screen.dart` and `fullmock_review.dart`.

```
QuerySnapshot querySnapshot = await _eventsCollectionRef
    .where("date", isEqualTo: Timestamp.fromDate(date))
    .get();

List<String> updatedEvents =
    allEvents[date]?.map((e) => e.title).toList() ?? [];

if (querySnapshot.docs.isNotEmpty) {
  String docId = querySnapshot.docs.first.id;
  await _eventsCollectionRef.doc(docId).update({"events": updatedEvents});
} else {
  DocumentReference newDoc = await _eventsCollectionRef.add({
    "date": Timestamp.fromDate(date),
    "events": [eventTitle]
  });
  await _eventsCollectionRef.doc(newDoc.id).update({"events": updatedEvents});
}
```

`deleteSingleEvent()` works the same way, then either `deletes` the doc (if nothing's left) or `updates` it.

1.5 Scoring page: read everything, save the score

`lib/screens/minimock_scoring.dart`

This reads the questions (same `queryQuestionData` as the exam page, not repeated), the user's answers, and the average score, then saves the result.

Read the saved answers. Find the user, then read the one document from `examAnswers` and pull out its `answers`:

```
Future<Map<String, dynamic>> queryUserAnswerData() async {
  QuerySnapshot snapshot = await FirebaseFirestore.instance
      .collection('users')
      .where('userID', isEqualTo: user.uid)
      .get();
  var id = snapshot.docs.last.id;

  DocumentSnapshot<Map<String, dynamic>> userAnswers =
      await FirebaseFirestore.instance
          .collection('users').doc(id)
          .collection('examAnswers').doc(widget.answerDirectory)
```

```
          .get();
  return userAnswers.get('answers');
}
```

Read the average score from the `mini-mock-stats` collection (just reading one document):

```
Future<void> fetchAverageScore() async {
  DocumentSnapshot<Map<String, dynamic>> snapshot =
      await FirebaseFirestore.instance
          .collection('mini-mock-stats')
          .doc(widget.questionDirectory)
          .get();
  int median;
  (snapshot.exists) ? median = snapshot.get('median') : median = 0;
  averageScore = median.toDouble();
}
```

Save the score back onto the same answers document. `update` only touches `score`, so the answers and timestamp stay put:

```
Future<void> _updateScore(int score) async {
  final CollectionReference users =
      FirebaseFirestore.instance.collection('users');
  QuerySnapshot u = await users.where("userID", isEqualTo: user.uid).get();
  var id = u.docs.last.id;

  DocumentReference userDocRef =
      FirebaseFirestore.instance.collection('users').doc(id);
  DocumentReference scoreDocRef =
      userDocRef.collection('examAnswers').doc(widget.answerDirectory);

  await scoreDocRef.update({'score': score});
}
```

1.6 Feedback page

`lib/screens/minimock_feedback.dart`

Nothing new here — it reads the questions and the user's answers exactly the same way as the scoring page, then just shows them. Good example of how the same two reads get reused all over the place.

2 What you might consider

Even though this is all in Flutter, the same patterns apply to any language. If you are set on using python you might want to have a quick look at the functions/main.py file in the repo. There are a few examples in there on using firebase in python. I imagine that you'd need to update the user document to store all the incorrect answers and the date a user got a question wrong— so you can use that information for your AI model. Besides that, you should mostly just be accessing data that our database already stores.